Product Requirements Document GitLab Build, Stable & Release Management Dashboard

Centralized commit-to-build-to-stable-to-release management for GitLab CI/CD.

Document	PRD - Build & Release Management
Version	1.0
Date	6 October 2026
Status	Proposed Internship Project
Primary Users	Development, QA, DevOps, Release Engineering, Engineering Management

Project principle: Do not rebuild GitLab CI/CD. Provide a simple, controlled workflow that lets engineers move safely from a selected commit to build, stable promotion and release, while reusing existing signing and repackaging capabilities.

1. Overview

The existing internal build dashboard provides a useful commit and build-oriented view, but the engineering workflow is moving to GitLab. A GitLab-native replacement is needed to support multiple products and branches while keeping the operational experience simple.

This project proposes a Build, Stable & Release Management Dashboard. It complements the existing GitLab Pipeline Monitoring Dashboard and is intentionally designed so both internship projects can later be integrated into one unified engineering dashboard.

2. Problem Statement

After CI/CD migration to GitLab, engineers need a controlled way to identify a commit, trigger a build, verify pipeline health, promote an eligible build to Stable, and create a release build. Without a consolidated workflow, users must navigate multiple projects, branches, pipelines and release tools, increasing operational effort and the risk of selecting the wrong build.

3. Project Objectives

- View commit history across configured GitLab products, projects and branches.
- Trigger a build from a selected commit through an authorized backend action.
- Show the corresponding GitLab pipeline status and provide direct drill-down links.
- Allow Stable promotion only when required validation conditions are satisfied.
- Update the configured Stable repository/reference after a successful promotion.
- Create a Release Build from an eligible Stable build.
- Invoke the existing signing and repackaging workflows and display their status.
- Maintain an auditable history of build, Stable and Release actions.
- Reuse the common architecture and UX patterns of the Pipeline Monitoring Dashboard to simplify final integration.

4. Scope

MVP - Must Have

- Configurable GitLab products/projects and multiple branches.
- Commit retrieval and searchable commit history.
- Build trigger from an eligible commit.
- Pipeline status tracking and GitLab drill-down.
- Backend validation before Stable promotion.
- Stable promotion and update of the configured Stable repository/reference.
- Build and promotion history with user/time/audit information.
- Authentication, authorization and secure GitLab/API credential handling.

Phase 2 / Release Flow

- Create Release Build only from an eligible Stable build.
- Integrate existing signing workflow.
- Integrate existing repackaging workflow.
- Track release, signing and repackaging status through completion.

Out of Scope for MVP

- Replacing GitLab source browsing, merge requests, job logs or administration.
- Reimplementing signing/repackaging engines that already exist.
- Artifact repository cleanup or retention management.

- Advanced release analytics and AI-based recommendations.

5. Target Users

User	Primary Need
Developers	Find a commit, trigger a build and check build/pipeline status.
QA / Automation	Confirm required validation completed before Stable promotion.
DevOps / CI-CD	Operate and troubleshoot build/promotion integrations safely.
Release Engineering	Promote Stable builds and initiate controlled release builds.
Engineering Managers	See build, Stable and release readiness across branches.

6. Core User Experience

The dashboard should make the build lifecycle obvious and require minimal navigation. The expected flow is:

Product / Branch → Commit → Build → Pipeline Validation → Promote Stable → Stable Repository → Release Build → Signing → Repackaging → Release Ready

Multi-branch experience

A user selects a product/component and then a branch or release stream. The screen refreshes the commit/build history for that context. The solution must not be designed only for main; release, patch and future branches must be configurable without UI code changes.

7. Functional Requirements

ID	Requirement	Description
FR-01	Product / Project Configuration	Configure multiple GitLab products/projects without hardcoding them in the UI.
FR-02	Branch Configuration	Configure and display multiple branches/release streams for each product.
FR-03	Commit Retrieval	Retrieve commit SHA, message, author, timestamp and branch from GitLab.
FR-04	Commit History	Display recent commits with search/filter and a direct GitLab commit link.
FR-05	Build Trigger	Allow an authorized user to trigger the configured GitLab build pipeline for a selected commit.
FR-06	Build Tracking	Store the relationship between product, branch, commit, build/version, pipeline and trigger user.
FR-07	Pipeline Status	Display pending/running/success/failed/canceled state and link to the GitLab pipeline/jobs.
FR-08	Stable Eligibility	Enable Stable promotion only after backend verification of configured mandatory conditions.
FR-09	Stable Promotion	Promote an eligible build to Stable and update the configured Stable repository/reference.
FR-10	Stable History	Identify current and previous Stable builds by branch and preserve promotion history.
FR-11	Release Eligibility	Allow Release Build creation only from an eligible/approved Stable build.
FR-12	Release Build	Trigger the configured release pipeline/workflow from the selected Stable build.
FR-13	Signing Integration	Invoke or monitor the existing signing process; do not reimplement signing in the dashboard.
FR-14	Repackaging Integration	Invoke or monitor the existing repackaging process and expose its result.
FR-15	Audit Trail	Record who triggered, promoted or released a build, with timestamp and outcome.

ID	Requirement	Description
FR-16	Authorization	Provide role-based permissions for Build, Promote Stable and Release actions.
FR-17	Duplicate Protection	Prevent duplicate or conflicting promotions/releases and make operations idempotent where practical.
FR-18	Errors / Retry	Show meaningful failures; support controlled retry only where the backend confirms it is safe.
FR-19	Integration API	Expose/reuse APIs and shared models suitable for later integration with Pipeline Monitoring.
FR-20	Activity History	Provide searchable history by product, branch, commit, version, status and date range.

8. Stable Promotion Guardrails

Promotion changes release state, so the backend must be the source of truth. UI state alone must never authorize promotion.

- Re-read the selected commit/build state from GitLab or the trusted backend record immediately before promotion.
- Confirm the build belongs to the selected product and branch.
- Confirm the configured mandatory pipeline/jobs completed successfully.
- Confirm required build artifacts exist and the build is not already promoted.
- Reject promotion if any eligibility rule fails and return a clear reason.
- Record the promotion request, actor, source build, result and resulting Stable version/reference.

9. Suggested Dashboard Fields

Field	Example / Purpose
Product / Component	ESM / SMAX / configured component
Branch / Release	main, release/26.4, patch/26.4.x
Commit Time	GitLab commit timestamp
Commit ID	Short SHA; clickable to GitLab
Change	Commit message / change summary
Build / Version	Generated version/build identifier
Pipeline	Pipeline ID and link
Pipeline Status	Pending / Running / Success / Failed / Canceled
Tags	Stable / Released
Actions	Build / Promote Stable / Release
Signing	Not Started / Running / Success / Failed
Repackaging	Not Started / Running / Success / Failed
Audit	Triggered/promoted/released by and timestamp

10. Current-State / Reference UI

The following current dashboard screenshot is a reference for the initial interaction model: left-side branch navigation, commit history, version/tags, and lifecycle actions such as Build, Promote Stable and Promote Released. The new implementation should modernize the experience rather than duplicate the legacy UI exactly.

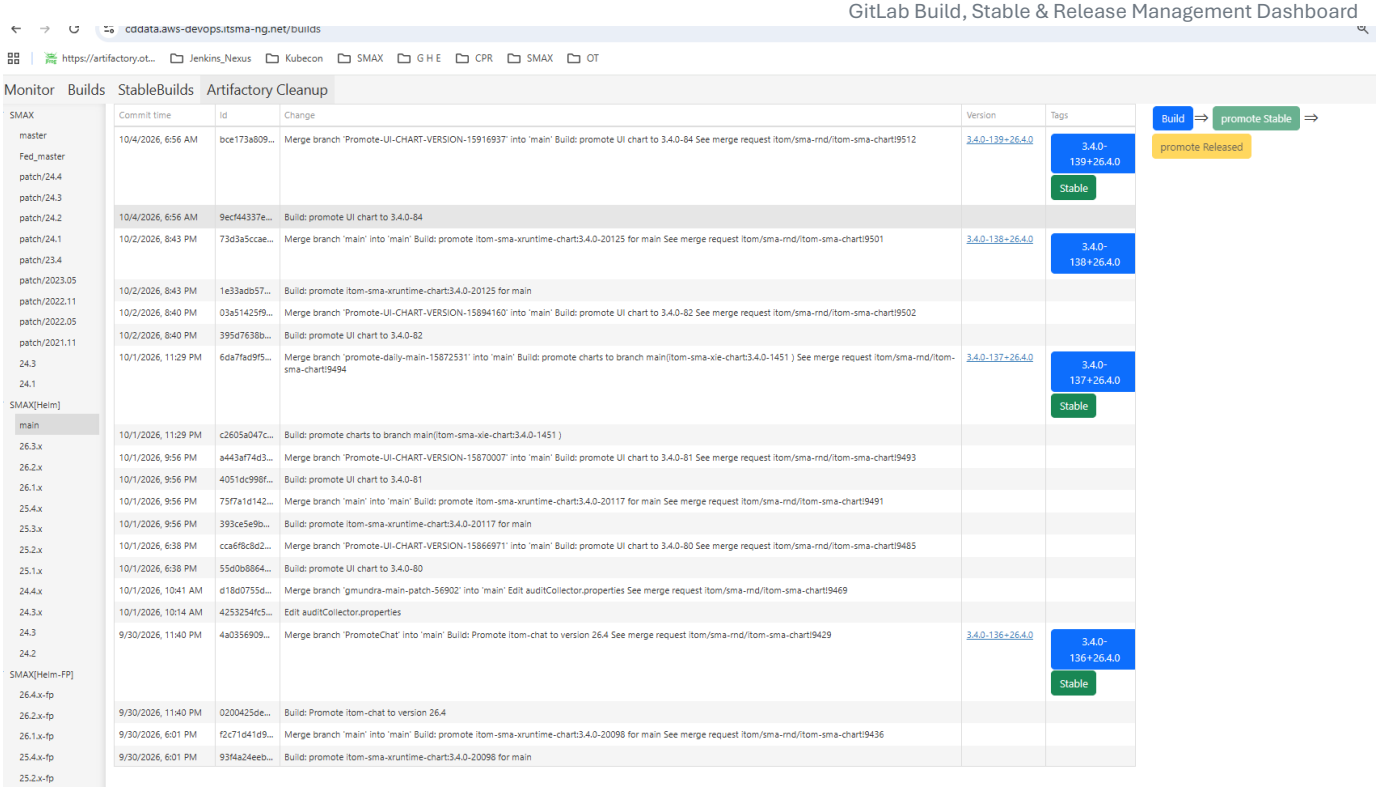


Figure 1. Current build dashboard reference provided for the PRD.

11. High-Level Architecture

Layer	Responsibility
GitLab	Source for repositories, branches, commits, pipelines, jobs and build status.
Backend Service	Authenticates requests, integrates with GitLab/existing release services, enforces eligibility, records audit events and exposes APIs.
Database	Stores configuration, build mappings, Stable/Release history, workflow state and audit trail.
Frontend	Product/branch navigation, commit/build view, lifecycle actions and status visualization.
Existing Signing / Repackaging	Existing release capabilities invoked or monitored by the dashboard; not reimplemented.
Shared Platform	Authentication, configuration conventions and integration points reused with Pipeline Monitoring where practical.

12. Integration with Pipeline Monitoring Dashboard

The Pipeline Monitoring PRD deliberately focuses its MVP on pipeline visibility and troubleshooting, keeps full Stable-build management out of scope, and identifies Stable Builds/release readiness as future enhancements. This second project fills that adjacent capability while preserving a clean separation of concerns.

Both projects should therefore converge on a common product experience rather than remain unrelated applications. Recommended final navigation:

- Overview: cross-product build and release health.
- Pipelines: pipeline/job monitoring and failure drill-down from Intern Project 1.
- Builds: branch, commit, build trigger and build history from Intern Project 2.
- Stable Builds: current/previous Stable state and controlled promotion.
- Releases: release build, signing, repackaging and release history.

Integration principles: reuse authentication, UI design system, API conventions, product/project configuration and environment/deployment approach wherever practical. Avoid duplicate GitLab ingestion and duplicate data models when a shared service or API can provide the same information.

13. Security and Reliability

- Never expose GitLab tokens, signing credentials or release credentials in the browser.
- Use approved secret management and least-privilege service/API access.
- Authenticate users and enforce authorization server-side for all lifecycle actions.
- Require stronger permissions for Promote Stable and Release than read-only access.
- Do not rely on hidden/disabled UI buttons as an authorization control.
- Validate all user-controlled identifiers and configured downstream endpoints.
- Keep an immutable or append-only audit record of lifecycle actions where practical.
- Make action APIs idempotent or duplicate-safe to reduce accidental repeat operations.
- Show last refresh/synchronization time and meaningful user-facing errors.

14. Suggested Internship Work Plan

This PRD assumes one intern can own the Build & Release Dashboard while collaborating with the Pipeline Monitoring intern on shared APIs and final integration. Scope should be staged so a production-quality Build → Stable flow is achieved before Release complexity is added.

Milestone	Deliverable
1. Current-state study	Review existing Builds UI, GitLab pipelines, existing Stable process, signing and repackaging interfaces.
2. GitLab/API proof of concept	Read products/branches/commits and trigger a safe test pipeline.
3. Technical design	Data model, API contract, permission model and Stable eligibility rules.
4. Build MVP	Branch/commit dashboard, build trigger, pipeline tracking and history.
5. Stable MVP	Backend validation, Promote Stable, configured Stable reference/repository update and audit.
6. Release integration	Release Build plus existing signing and repackaging integration.
7. Hardening	Authorization, duplicate protection, negative tests, error/retry handling and audit validation.
8. Unified integration/demo	Connect with Pipeline Monitoring navigation/APIs and demonstrate end-to-end lifecycle.

15. Definition of Done

- Multiple products/projects and branches can be configured without UI code changes.
- Recent commits can be viewed and searched for a selected branch.
- Authorized users can trigger a build for a selected commit.
- Triggered builds are linked to the correct commit, branch and GitLab pipeline.
- Stable promotion is rejected unless backend eligibility checks pass.
- Successful promotion updates the configured Stable repository/reference and is visible in history.
- Release Build can be initiated only from an eligible Stable build when Phase 2 is enabled.
- Existing signing and repackaging status can be tracked without embedding sensitive credentials in the UI.
- Build, promotion and release actions are auditable by user and timestamp.
- Basic automated tests, deployment automation, operational documentation and final demo are complete.
- The solution can be integrated with Pipeline Monitoring without redesigning core navigation or data contracts.

16. Success Measures

- Engineers can move from branch/commit selection to a traceable build without manually navigating multiple GitLab pages.
- Stable promotion follows consistent server-side eligibility rules and records a complete audit trail.
- Release Engineering can identify the current Stable build and release status quickly for each supported branch.
- The two internship projects demonstrate a credible unified GitLab Engineering Dashboard rather than two disconnected tools.

17. Future Enhancements

- Artifact repository and artifact metadata visualization.
- Release readiness checks combining pipelines, automated tests and deployment health.
- Approval workflows for high-risk promotions/releases.
- Notifications for Stable/Release lifecycle events.
- Trend analytics for build success, promotion frequency and release lead time.
- AI-generated explanations only after the underlying lifecycle and audit data are reliable.